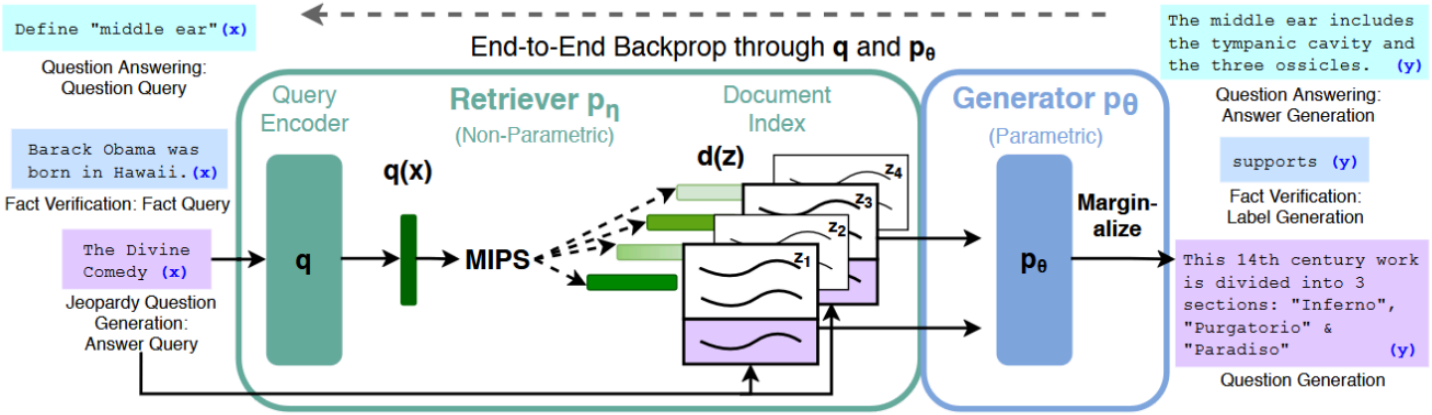
SI3003 - Inteligencia Artificial

CLASE 10 – RETRIEVAL AUGMENTED GENERATION (RAG)



Lewis et al., *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (2020).

Agenda

1. Modelos de Lenguaje
2. RAG
3. Embeddings

Parte 1 — Modelos de Lenguaje

LLMs

Un modelo de lenguaje es un **modelo probabilístico** que asigna probabilidades a secuencias de palabras.

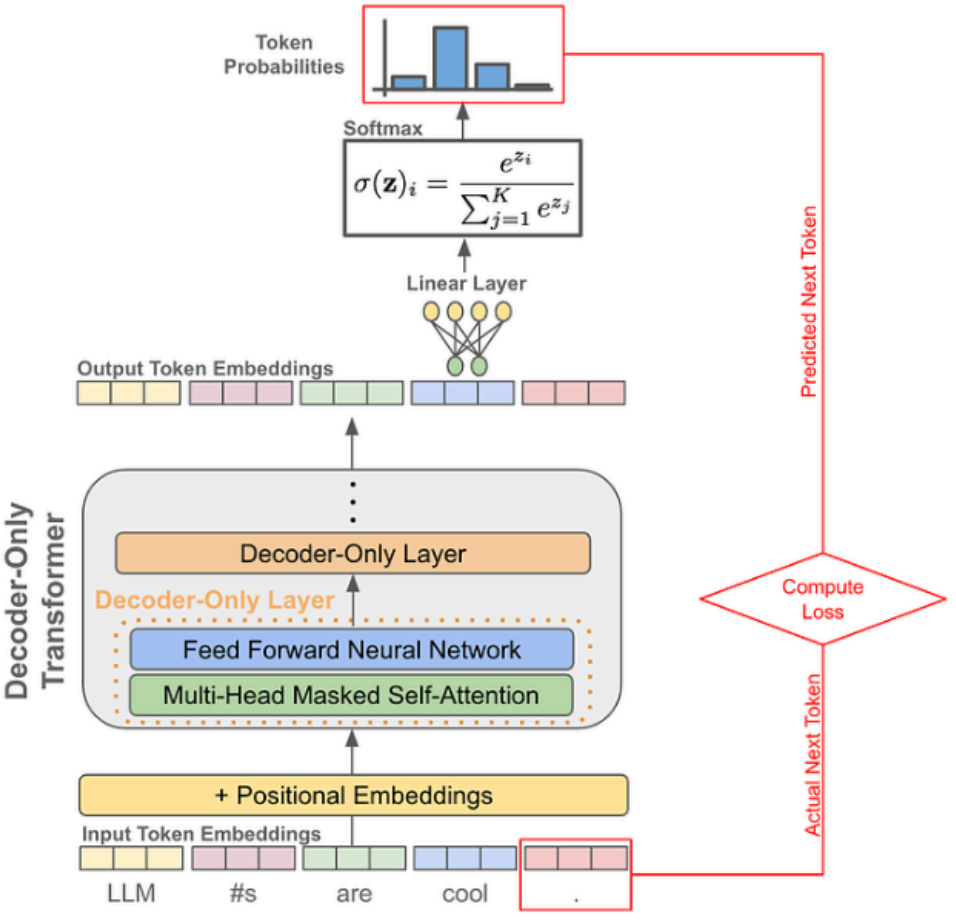
En la práctica: dado un texto de entrada (*prompt*), el modelo predice cuál es el siguiente token más probable — y repite este proceso para generar la respuesta completa.

Entrenamiento

Se entrena en enormes corpus de texto (Wikipedia, libros, web). El modelo aprende **patrones estadísticos** del lenguaje.

Inferencia

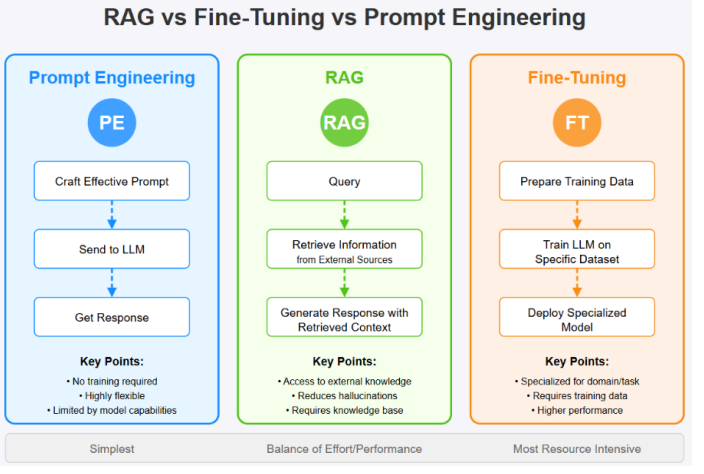
Dado un prompt, genera tokens uno a uno hasta completar la respuesta. Es **autoregresivo** — cada token depende de los anteriores.


Token Probabilities
Softmax
$\sigma(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}}$
Linear Layer
Output Token Embeddings
Decoder-Only Transformer
Decoder-Only Layer
Decoder-Only Layer
Feed Forward Neural Network
Multi-Head Masked Self-Attention
+ Positional Embeddings
Input Token Embeddings
LLM
#s
are
cool
.
Predicted Next Token
Compute Loss
Actual Next Token

Problema - Contexto

El LLM solo sabe lo que vio en el entrenamiento:

- No conoce tus **documentos internos**
- No conoce **eventos recientes** (tiene fecha de corte)
- No puede acceder a **datos en tiempo real**
- Puede **"alucinar"** — inventar información plausible pero falsa

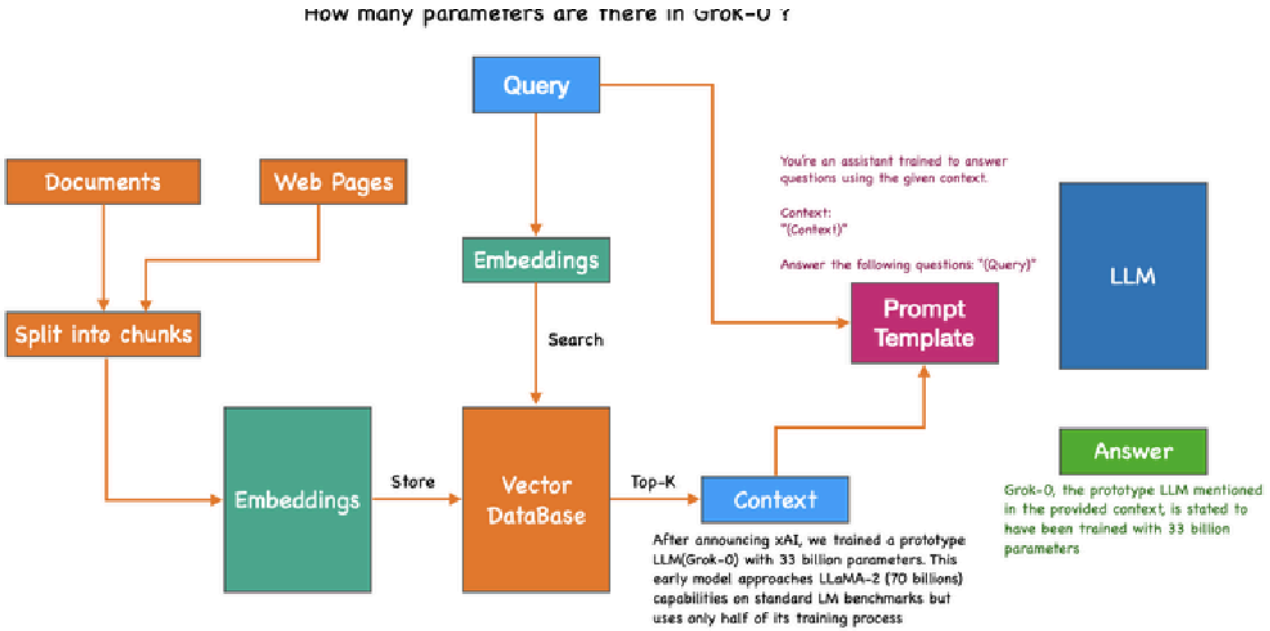


EAFIT

UNIVERSIDAD

Parte 2 — RAG

Retrieval Augmented Generation - RAG



Desde primeros principios: RAG es una función donde, dada una pregunta del usuario, la convertimos en vectores, buscamos los vectores más similares en nuestra base de datos, y concatenamos los chunks recuperados al prompt. Así de simple.

¿Por qué Chunking?

No podemos meter documentos completos en cada query — hay dos razones principales.

Ventana de contexto

Los LLMs tienen un límite de tokens por request. Un documento de 100 páginas no cabe. Los chunks resuelven esto.

Calidad del retrieval

Un chunk que habla de un solo tema genera un vector más representativo. Documentos largos mezclan temas y degradan la búsqueda.

Chunking semántico respeta párrafos y oraciones — produce chunks con significado cohesivo.

Tamaño del chunk

Tamaño	Problema
Muy pequeño	Falta contexto, chunks sin sentido completo
Muy grande	Vectores diluidos, retrieval impreciso
~400-600 chars	Balance óptimo para la mayoría de casos

Tipos de Búsqueda

Vectorial

Busca por **similitud semántica**. Entiende sinónimos y contexto.

⚠ Puede devolver resultados relacionados pero no exactos

Keyword (BM25)

Coincidencias **exactas** de términos. Ideal para nombres propios y siglas.

⚠ No entiende relaciones semánticas

Híbrida ← producción

Combina vectorial + keyword. Reordena por scores combinados.

✅ Mejor recall y precision en conjunto

Ejemplo: buscar "*política de vacaciones*" con búsqueda vectorial puede devolver "*días de descanso*" (semánticamente similar pero sin la palabra exacta). BM25 lo perdería. Híbrida encuentra ambos.

EAFIT

UNIVERSIDAD

Parte 3 — Embeddings

Embeddings

Un **embedding** es una función que convierte texto en un vector de números reales. Textos semánticamente similares quedan

cerca en el espacio vectorial

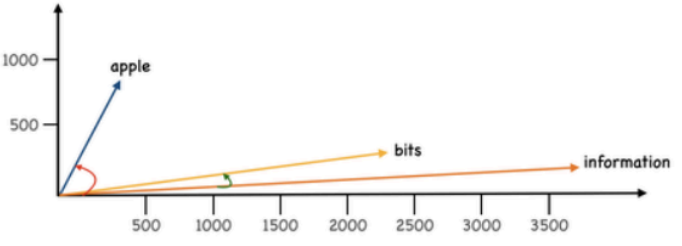
.

Hipótesis distribucional: palabras que aparecen en el mismo contexto tienden a tener significados similares. *"teacher"* y *"professor"* suelen aparecer junto a *"school"*, *"exam"*, *"course"*.

Self-Attention en Transformers captura exactamente esto — relaciona cada token con todos los demás de la oración.

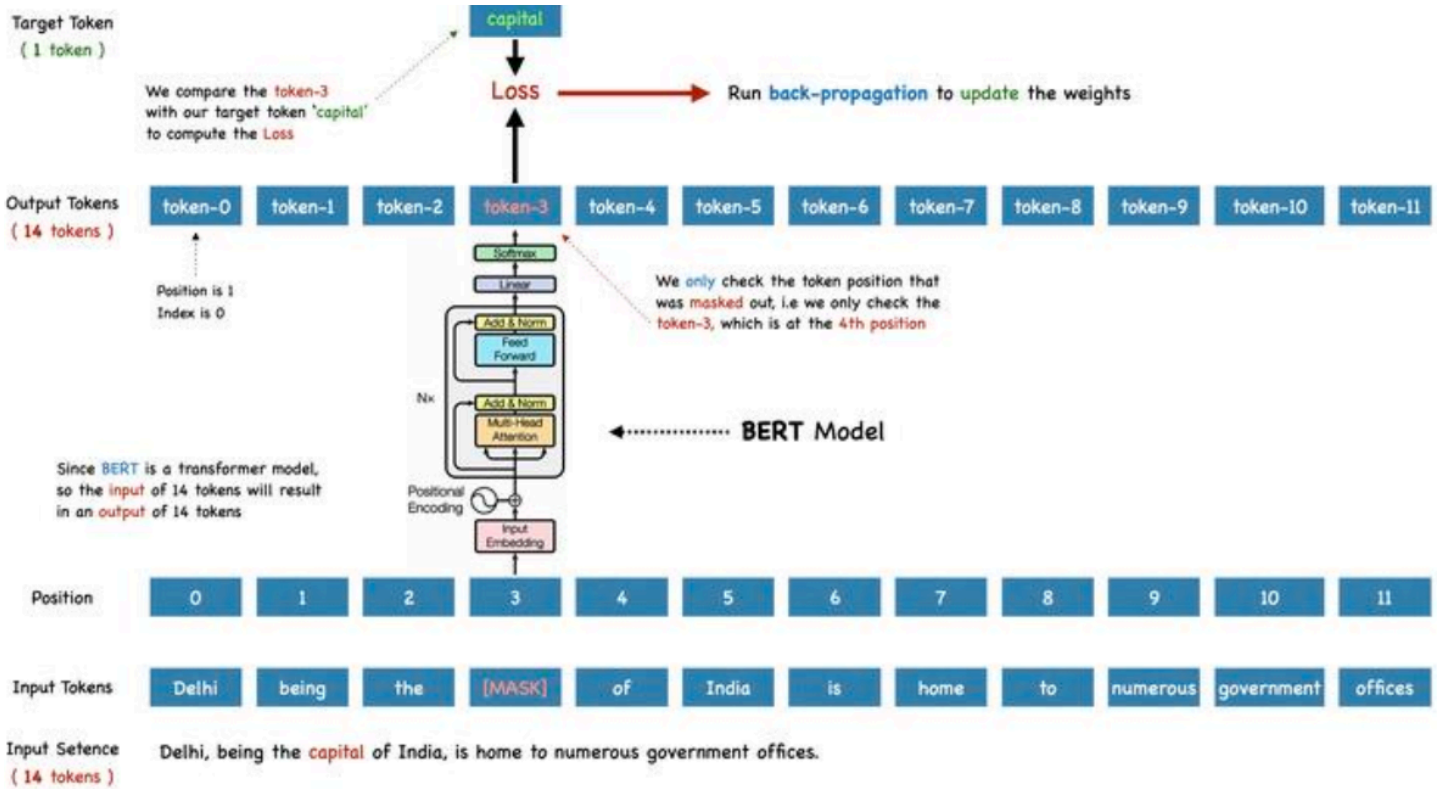
Similitud del coseno

```
cos(A,B) = dot(A,B) / (‖A‖·‖B‖) # Rango: -1 (opuestos)
→ 1 (idénticos) # 0 = ortogonales (sin relación)
sim("gato", "felino") → 0.89 sim("gato", "reembolso")
0.12
```



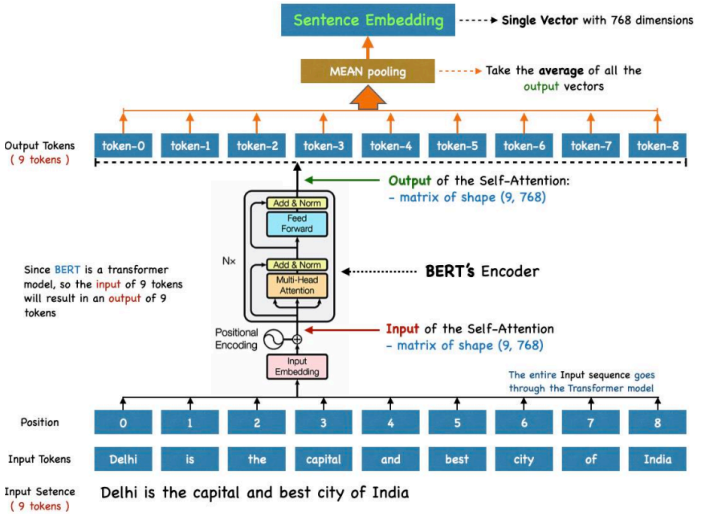
Rango: -1 (opuestos) → 1 (idénticos); 0 = ortogonales (sin relación).

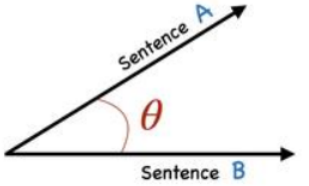
BERT



Self-Attention captura el "significado" de una oración completa.

Embeddings de Oraciones



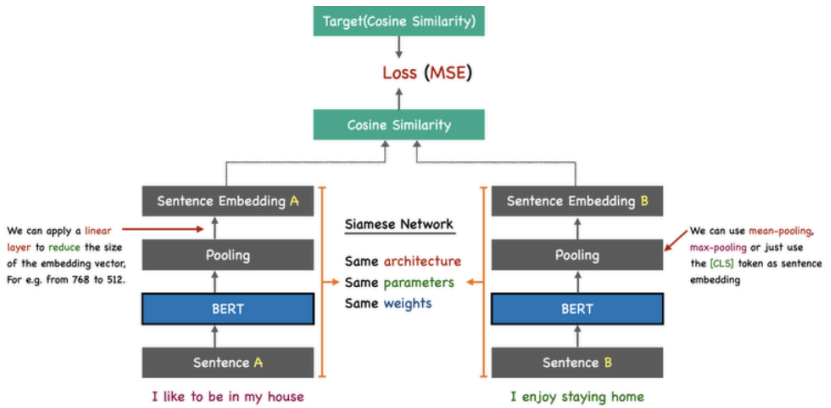


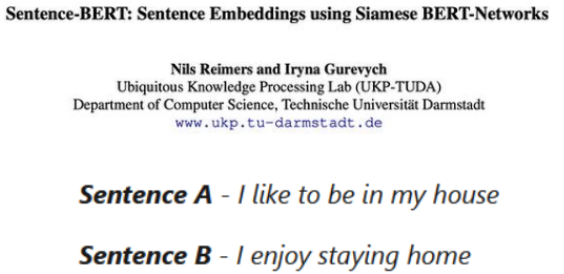
$$cosine\ similarity = S_C(A,B) = cos(\theta) = \frac{\mathbf{A}.\mathbf{B}}{\|\mathbf{A}\|\|\mathbf{B}\|} = \frac{\sum_{i=1}^{n} A_i B_i}{\sqrt{\sum_{i=1}^{n} A_i^2}\sqrt{\sum_{i=1}^{n} B_i^2}}$$

La entrada de 9 tokens produce 9 vectores de salida de 768 dimensiones. El **MEAN pooling** promedia todos esos vectores y entrega un **único vector de 768 dimensiones** que representa la oración completa.

Sentence-BERT

Problema: nadie le dijo a BERT que los embeddings que produce deberían ser comparables con la similitud del coseno, es decir, que dos oraciones similares deberían estar representadas por vectores que apunten en la misma dirección en el espacio.

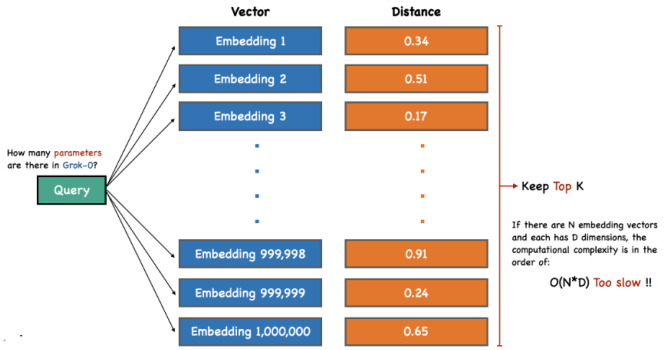




Base de datos vectorial

Vector DB: almacena embeddings (vectores de dimensión fija)

- **Objetivo:** encontrar elementos similares
- **Similitud:** coseno o distancia euclidiana
- **Método:** KNN / búsqueda aproximada (ANN)
- **Uso:** recomendación y búsqueda
- **Valor:** recuperación semántica rápida a escala



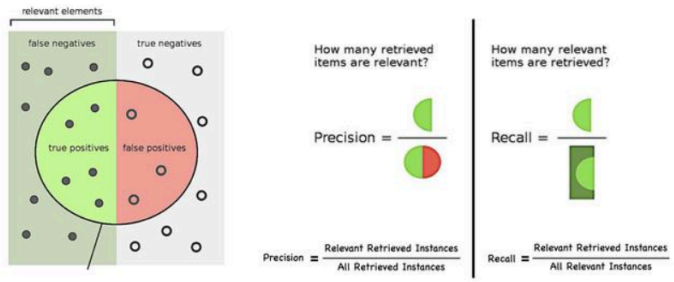
Con 1M de chunks de 768 dims: 768 millones de operaciones por query. Demasiado lento para producción.

ANN: Cambiar precisión por velocidad

La idea: reducir el número de comparaciones pero obtener resultados precisos con **alta probabilidad**.

Métricas que importan:

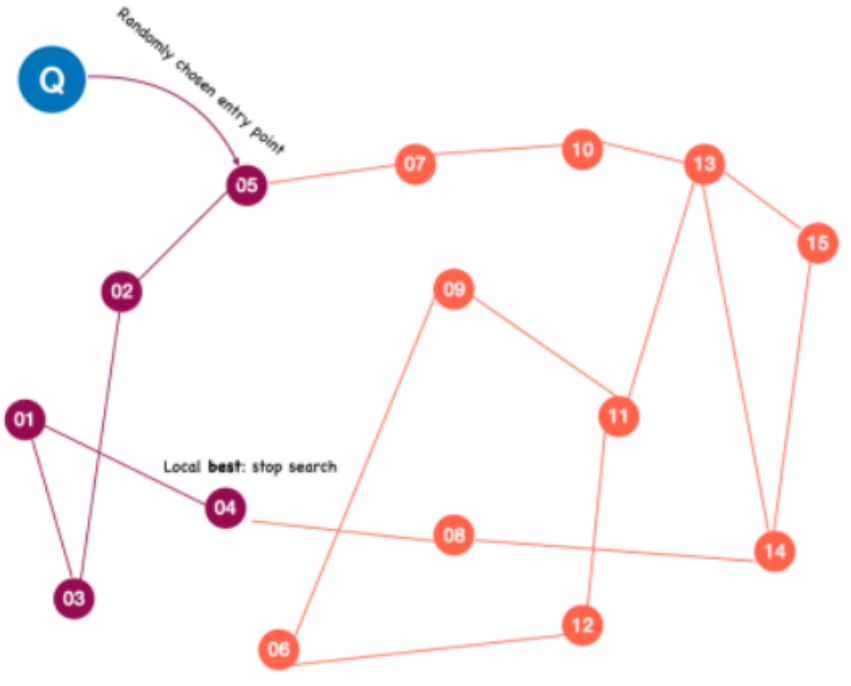
Métrica	Qué mide
Recall	¿Qué % de los K vecinos reales encontramos?
Latencia	Tiempo de respuesta por query
Throughput	Queries por segundo



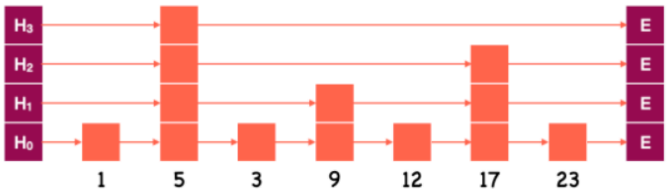
En RAG nos importa principalmente el **recall** — si perdemos un chunk relevante, la respuesta puede ser incorrecta.

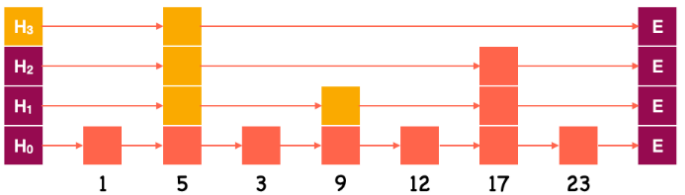
NSW: Navigable Small World

Node	*Text*
01	*The transformer is a model*
02	*Diagnose tumor with AI*
03	*A transformer-based model*
04	*Transformer has 12 layers*
05	*An MRI costs 10000$*
06	*The dot product is based on cosine function*
07	*Big-Pharma is not very big*
08	*Cross-Attention is a grate attention method*
09	*To solve an ODE you need math*
10	*We are aging too fast because of disease*
11	*Open-Source models are free*
12	*Bert is a two phase model*
13	*AI can control aging in future*
14	*Attention is all you need is of Google*
15	*LLaMA-2 has & billions parameters*



HNSW: Hierarchical Navigable Small World

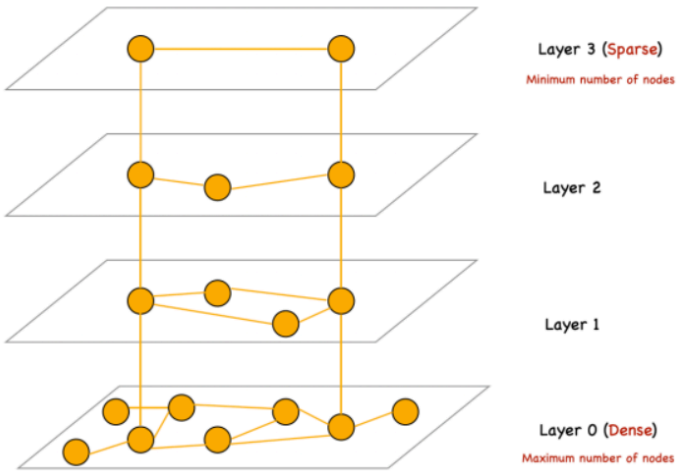


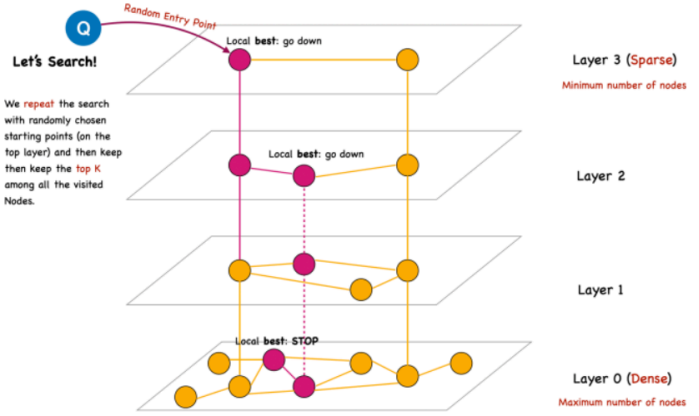




HNSW combina la **jerarquía por capas** de las *skip lists* con la **navegabilidad** de los grafos *small world*.

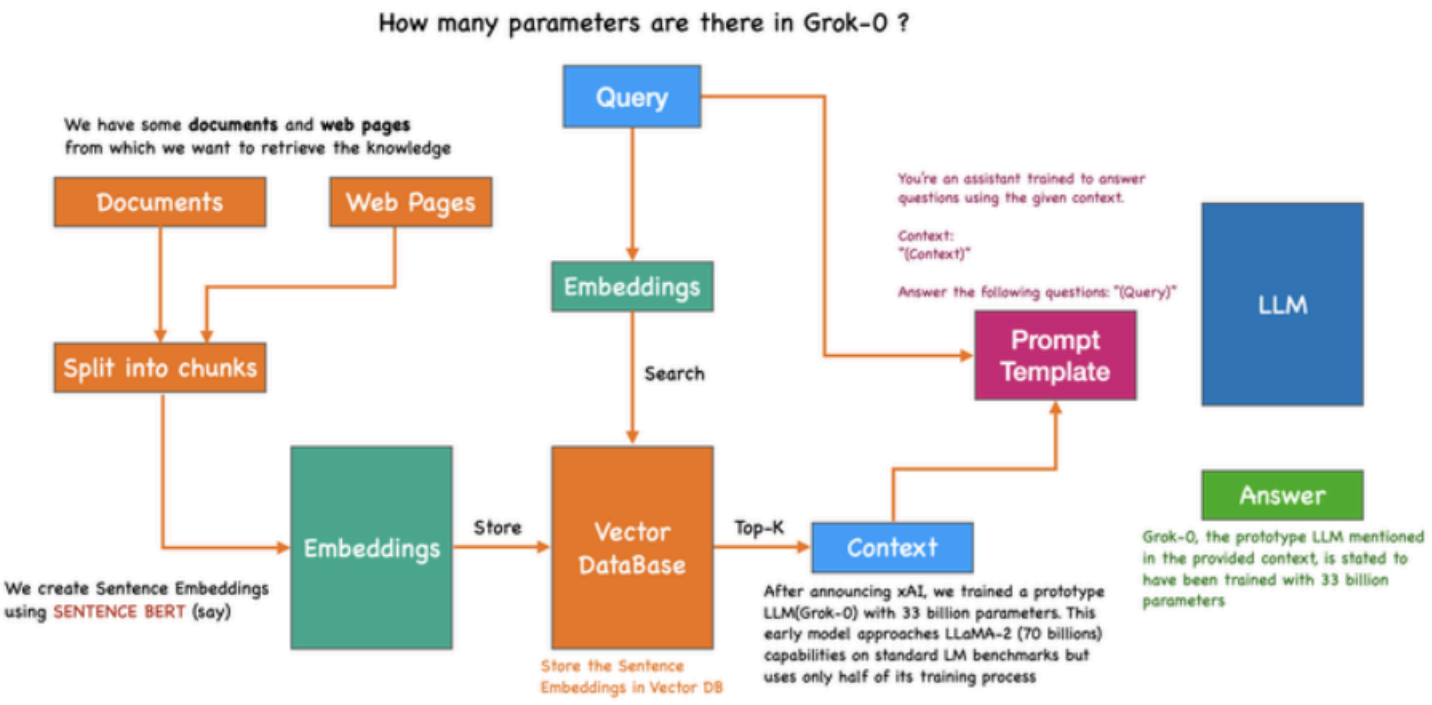
HNSW: Hierarchical Navigable Small World





Se repite la búsqueda con puntos de partida aleatorios en la capa superior y se conserva el **top-K** entre todos los nodos visitados.

RAG Pipeline



Evaluación de RAG: Faithfulness y Answer Relevancy

Faithfulness — fidelidad al contexto

¿Las afirmaciones de la respuesta están respaldadas por los chunks recuperados?

$$F = \frac{\text{afirmaciones respaldadas}}{\text{afirmaciones totales}}$$

Un LLM extrae las afirmaciones y verifica cada una contra el contexto.

Pregunta: ¿A qué hora cierra la biblioteca?
Contexto: «Cierra a las 8 p. m. y tiene tres pisos».

Answer Relevancy — pertinencia

¿La respuesta atiende la pregunta del usuario?

En Ragas: un LLM genera preguntas a partir de la respuesta; se promedia su similitud del coseno con la pregunta original usando embeddings.

Resumen

- Un **LLM** es un modelo probabilístico autoregresivo: predice el siguiente token y repite. Todo su conocimiento vive en los pesos — no conoce tus documentos, ni eventos posteriores a su fecha de corte, y cuando no sabe puede *alucinar*.
- **RAG** no es una arquitectura nueva: es convertir la pregunta en un vector, buscar los chunks más similares y concatenarlos al prompt. Da acceso a conocimiento externo sin reentrenar nada.
- El **chunking** existe por dos razones distintas: la ventana de contexto (restricción dura) y la calidad del retrieval (un vector por chunk, así que un chunk = un tema). ~400–600 caracteres es un punto de partida razonable.
- La **búsqueda híbrida** es el estándar de producción: la vectorial aporta semántica, BM25 aporta coincidencias exactas de siglas y nombres propios.
- Un **embedding** convierte texto en un vector donde la **similitud del coseno** mide parecido semántico. La hipótesis distribucional es el supuesto de fondo.
- **BERT** da un vector por token; el **mean pooling** los promedia en uno solo. Pero **Sentence-BERT** es el que entrena explícitamente para que ese vector sea comparable con coseno — red siamesa, mismos pesos, costo MSE.
- La búsqueda exacta en una **base de datos vectorial** es $O(N \cdot D)$ e inviable a escala. Por eso se usa **ANN**: se cambia exactitud por velocidad, midiendo **recall**, latencia y throughput.
- **HNSW** = skip list + navigable small world: capas dispersas arriba para cubrir el espacio con saltos grandes, capa densa abajo para refinar. Es el índice que usan las vector DB modernas.

EAFIT

UNIVERSIDAD

Fin de la Clase 10

Próxima clase: Agentes modernos con LLMs